

# NeuroMap3D: Vectorized Neuronal Space Mapping Pipeline

Technical Documentation

March 29, 2026

## 1 Project Title

**NeuroMap3D:** A Vectorized Pipeline for 3D Neuronal Probability Mapping.

## 2 Description

This function is a highly optimized, vectorized pipeline designed to map the physical 3D space occupied by a population of neurons. It converts discrete, 1D point-and-radius data into a continuous 3D probability map, identifying the likelihood of finding an axon or dendrite in any given micro-region of space.

## 3 Getting Started

The pipeline operates by processing raw neuronal morphological data (points and radii) and projecting them onto a standardized spatial grid. The workflow is divided into initialization, alignment, interpolation, jittering, and rasterization.

## 4 Prerequisites

To run this pipeline, the following environment is required:

- **Python 3.8+**
- **NumPy:** For vectorized matrix operations and `histogramdd`.
- **Pandas:** For handling neuronal point-cloud dataframes.
- **SciPy:** For spatial rotations and nearest-neighbor calculations.

## 5 Technical Workflow

### 5.1 Step 1: Initialization and Grid Setup

The script begins by preparing the canvas:

- **Metadata Loading:** Loads reference files containing soma coordinates and rotation matrices to ensure population alignment.

- **Grid Creation:** Uses `np.arange` to build an invisible 3D bounding box (the grid) defined by `grid_extent` and `voxel_size`.
- **Master Accumulators:** Initializes `axon_presence_total` and `dend_presence_total` as zero-arrays.

## 5.2 Step 2: Neuron Loading and Alignment (Rototranslation)

For each neuron, positions are standardized:

- **Soma Centering:** Every point is translated so the soma sits at (0, 0, 0).
- **Rotational Alignment:** Applies an averaged rotation matrix from  $k$ -nearest neighbors to align the neuron anatomically.
- **Unit Conversion:** Coordinates are converted from nanometers to micrometers.

## 5.3 Step 3: Vectorized Centerline Interpolation

To prevent gaps in the voxel grid, the function performs linear interpolation:

- **Calculating Point Density:** Based on `step_size = voxel_size / 3.0`, the pipeline calculates necessary intermediate points.
- **Matrix Math:** Generates fractional multipliers ( $t$  values from 0.0 to 1.0) to calculate exact coordinates for thousands of points simultaneously.

## 5.4 Step 4: Dynamic Geometric Jitter

To account for the 3D volume (tubular structure) of branches:

- **Thin Branches:** Uses a 7-point cross stencil for branches smaller than the voxel size.
- **Thick Branches:** Employs dynamic interpolation to generate intermediate radial points, ensuring no "holes" appear in the rasterization.

## 5.5 Step 5: Binary Histogram Rasterization

Spatial points are mapped to the discrete voxel grid:

- **Binning:** `np.histogramdd` assigns every point to a voxel boundary.
- **Binarization:** Applies a presence mask ( $> 0$ ). Any voxel touched by a neuron is marked as 1, regardless of point density within that voxel.

## 5.6 Step 6: Final Probability Calculation

The master arrays are divided by the total number of valid neurons processed to determine the final probability:

$$Probability = \frac{N_{\text{neurons present in voxel}}}{N_{\text{total valid neurons}}} \quad (1)$$

## 6 Usage

Input data should be provided as a structured dataframe containing  $x, y, z$ , and *radius* columns. Ensure `grid_extent` is set wide enough to encompass the full population.

## 7 License

This project is licensed under the MIT License.

## 8 Contact

For technical inquiries regarding the vectorized math or grid implementation, please contact the lead developer.